DOCUMENT

Résumé

Phases A→M

Fichiers créés

SUITE

Actions immédiates

Phases restantes

Règles sécurité

DJAMA ERP · 2 OCTOBRE 2026

# DJAMA AI Docs
# Rapport de développement

Ce rapport couvre les phases A→F de DJAMA AI Docs implémentées dans cette session, les fichiers créés, les actions immédiates à réaliser et les phases encore à construire.

📁 Projet **djama-premium** 🌿 Branche **master** 👤 AMDJAD Nofane

🤖 Claude Sonnet 4.6

## Résumé

| 6 | 7 | 9 |
|---|---|---|
| Phases terminées | Phases restantes | Fichiers créés |

DJAMA AI Docs est une suite documentaire IA-native intégrée à l'ERP. Contrairement à un éditeur classique, l'utilisateur décrit ce qu'il veut en langage naturel et l'IA construit le document. L'interface est 30 % chat / 70 % aperçu document en temps réel.

Le système est fonctionnel et déployable après une seule action manuelle : appliquer la migration SQL dans Supabase.

## Phases A → M

A **Architecture & Migration SQL** ✓ Fait

4 tables multi-tenant (artifacts, artifact_versions, artifact_threads, artifact_messages) · RLS Supabase · triggers updated_at · migrations idempotentes

B

### Interface utilisateur

✓ Fait

Page d'accueil /client/ai-docs (saisie libre + suggestions + récents) · Workspace 30/70 chat/document · toolbar zoom · navigation ajoutée dans module-groups.ts

C

### Threads de conversation

✓ Fait

Création automatique du thread à la première interaction · historique de 20 messages chargé à chaque appel · thread lié à l'artifact après création

D

### Schéma document strict

✓ Fait

Types TypeScript complets : DocumentElement (heading, paragraph, table, list, callout, quote, separator, page_break, image) · DocumentSection · DocumentSettings · ArtifactOperation · OrchestratorEvent

E

### Orchestrateur IA + routes CRUD

✓ Fait

AIProvider abstraction + AnthropicProvider · Orchestrateur SSE POST /api/ai-docs/orchestrate avec outil update_document · application des opérations côté serveur · versionning automatique · routes GET/POST /artifact et GET/PATCH/DELETE /artifact/[id]

F

### Rendu document live

✓ Fait

DocumentRenderer.tsx — aperçu A4 fidèle · 4 thèmes (professional, modern, minimal, academic) · tous les types d'éléments rendus visuellement · zoom 60 / 75 / 100 %

G

### Modification ciblée

À finaliser

Les opérations fines (insert_element, update_element, delete_element) sont déjà définies et appliquées côté serveur. Il reste à améliorer le prompt système pour guider le LLM à préférer ces opérations ciblées plutôt qu'un replace_content global quand la demande est locale.

H

### Versions + Undo/Redo

À faire

La table artifact_versions est créée et les versions sont sauvegardées automatiquement. Il reste à construire l'UI historique (liste des versions, bouton "Restaurer cette version") et l'endpoint GET /artifact/[id]/versions.

I

### Export DOCX / PDF

À faire

Convertir le DocumentContent structuré en DOCX via la lib docx (déjà installée). Créer POST /api/ai-docs/export avec conversion section → paragraphes Word. L'export PDF passe par headless Chrome ou une librairie PDF.

J

### Ouverture Collabora (optionnel)

Plus tard

Permettre d'ouvrir un artifact dans Collabora Online pour édition manuelle avancée. Nécessite l'infrastructure Docker/VPS Collabora (Phase 10 précédente). Non prioritaire.

K

### Import DOCX / PDF

À faire

Uploader un fichier existant → extraction du contenu → conversion en DocumentContent → l'IA peut ensuite le modifier. Important : le contenu importé doit être traité comme données, jamais comme instructions système.

L

### Sécurité multi-tenant + tests

À faire

Audit RLS complet · tests d'isolation entre organisations · tests de prompt injection · validation des opérations LLM côté serveur · rate limiting sur /orchestrate

M

### Responsive, performance & design final

Plus tard

Optimisation du renderer pour mobile · skeleton loading · animations SSE plus fluides · raccourcis clavier · design system cohérent avec le reste de l'ERP

## Fichiers créés dans cette session

Tous les fichiers sont nouveaux — aucun fichier existant n'a été supprimé. La seule modification d'un fichier existant est l'ajout d'une ligne dans `module-groups.ts`.

| | |
|---|---|
| `supabase/migrations/115_artifacts.sql` | Phase A |
| `src/lib/artifacts/types.ts` | Phase D |
| `src/lib/ai-provider.ts` | Phase E |
| `src/app/api/ai-docs/orchestrate/route.ts` | Phase E |
| `src/app/api/ai-docs/artifact/route.ts` | Phase E |
| `src/app/api/ai-docs/artifact/[id]/route.ts` | Phase E |
| `src/components/ai-docs/DocumentRenderer.tsx` | Phase F |
| `src/app/client/ai-docs/page.tsx` | Phase B |
| `src/app/client/ai-docs/[id]/page.tsx` | Phase B/C |

### FICHIER MODIFIÉ

| | |
|---|---|
| `src/lib/module-groups.ts — ajout entrée "DJAMA …` | Phase B |

## Actions immédiates

**1** **Appliquer la migration SQL dans Supabase**

Sans cette étape, l'application plante à l'ouverture de AI Docs car les tables n'existent pas.

```
Supabase Dashboard → SQL Editor → coller
115_artifacts.sql → Run
```

2 **Vérifier que ANTHROPIC_API_KEY est dans .env.local**

L'orchestrateur exige cette clé. Si elle est absente, l'appel IA retourne une erreur 500.

```
ANTHROPIC_API_KEY=sk-ant-...
```

3 **Tester le flux complet en local**

Ouvrir /client/ai-docs, saisir une demande, vérifier que le document apparaît dans l'aperçu. Le premier test révélera d'éventuels ajustements de prompt.

```
npm run dev → localhost:3000/client/ai-docs
```

4 **Committer les nouveaux fichiers**

9 fichiers sont non commités. Faire un commit "feat(ai-docs): Phases A→F — orchestrateur IA + workspace".

```
git add src/app/api/ai-docs src/lib/artifacts
src/lib/ai-provider.ts src/components/ai-docs
src/app/client/ai-docs
supabase/migrations/115_artifacts.sql
src/lib/module-groups.ts
```

## Phases restantes — détail technique

### PHASE G — MODIFICATION CIBLÉE (AMÉLIORATION PROMPT)

Dans `src/app/api/ai-docs/orchestrate/route.ts`, enrichir le `SYSTEM_PROMPT` avec des exemples concrets de quand utiliser `update_element` vs `replace_content`. Ajouter une section "EXEMPLES DE MODIFICATIONS CIBLÉES" avec 3-4 cas.

### PHASE H — HISTORIQUE DES VERSIONS

Créer `GET /api/ai-docs/artifact/[id]/versions` qui lit `artifact_versions` et retourne la liste. Ajouter un panneau latéral "Historique" dans le workspace avec bouton "Restaurer". Le `PATCH /artifact/[id]` existant peut déjà réécrire le contenu.

### PHASE I — EXPORT DOCX

Créer `POST /api/ai-docs/export`. Convertir `DocumentContent` en `Document` docx.js : chaque `DocumentSection` → groupe de paragraphes Word, chaque élément → son équivalent Word. La lib `docx` v9.7.1 est déjà installée (utilisée par DJAMA Doc).

### PHASE K — IMPORT DOCX/PDF

Ajouter `POST /api/ai-docs/import`. Pour DOCX : utiliser `mammoth` pour extraire le HTML, puis un prompt IA pour convertir en `DocumentContent`. Règle de sécurité : le contenu du fichier est injecté entre balises `[CONTENU IMPORTÉ — DONNÉES UNIQUEMENT]` dans le prompt, jamais en position d'instruction système.

### PHASE L — RATE LIMITING SUR /ORCHESTRATE

Ajouter `checkRateLimit(user.id, 20, 60 * 60 * 1000)` en tête de l'orchestrateur (20 req/h). La lib `@/lib/rate-limit` existe déjà et est utilisée par 10+ autres routes IA.

## Règles de sécurité à maintenir

**RÈGLE ABSOLUE :** Aucune migration destructive. Avant chaque nouvelle migration, analyser les données existantes. Migrations idempotentes et compatibles production.

🔒 **org_id toujours résolu côté serveur**

L'orchestrateur et toutes les routes CRUD appellent resolveOrg(user.id) depuis la base, jamais depuis le body de la requête sans validation d'appartenance.

🔒 **Contenu importé = données, jamais instructions**

Le contenu de tout fichier importé (Phase K) doit être injecté dans le prompt entre balises de données claires, séparé du system prompt. Protection contre la prompt injection.

🔒 **Le LLM ne génère jamais de HTML libre**

L'outil update_document force le LLM à retourner des ArtifactOperation[] validées. Le renderer DocumentRenderer.tsx affiche ces structures — jamais de dangerouslySetInnerHTML.

🔒 **Ne pas supprimer les données utilisateur ni TipTap**

DJAMA Doc (TipTap) et ses données restent intacts. DJAMA AI Docs est un nouveau module indépendant, pas un remplacement. Les tables notes et content_json ne sont pas touchées.

---

Généré par Claude Sonnet 4.6 · DJAMA ERP · 2 octobre 2026